

Cálculo Lambda

Basile (adaptado de Raúl Rojas)

15 de agosto de 2025

- Trata-se da menor linguagem de programação universal do mundo.
- Introduzido em 1930 por Alonzo Church.
- É universal no sentido de que qualquer função computável pode ser expressa e avaliada usando este formalismo.
- É equivalente a uma máquina de Turing.
- Diferentemente da máquina de Turing, enfatiza o uso de regras de transformação, não se importando com a máquina que o implementa.
- Trata-se, portanto, de um tratamento mais relacionado ao software que ao hardware.

$\langle \text{expressão} \rangle := \langle \text{nome} \rangle \mid \langle \text{função} \rangle \mid \langle \text{aplicação} \rangle$

$\langle \text{função} \rangle := \lambda \langle \text{nome} \rangle . \langle \text{expressão} \rangle$

$\langle \text{aplicação} \rangle := \langle \text{expressão} \rangle \langle \text{expressão} \rangle$

- Um exemplo de função é a função identidade:

$$\lambda x.x$$

- O nome após o λ é o identificador do argumento desta função.
- A expressão após o ponto (neste caso um único x) é chamado **corpo** da definição.

- Funções podem ser aplicadas a expressões:

$$(\lambda x.x)y$$

- Todas as ocorrências do identificador x no corpo da definição são substituídos por y .
- Os nomes dos argumentos nas definições de função não carregam qualquer significado. Ou seja:

$$\lambda z.z \equiv \lambda y.y \equiv \lambda x.x \equiv \lambda a.a$$

Variáveis livres e ligadas

- No cálculo λ todos os nomes são locais às definições.
- Na função $\lambda x.x$ dizemos que o x é **ligado** se todas as suas ocorrências no corpo for precedida por λx .
- Um nome não precedido por um λ é chamado **variável livre**.
- Na expressão abaixo a variável x é ligada e y é livre.

$$(\lambda x.xy)$$

- Na expressão abaixo o x no corpo da primeira expressão é ligado ao primeiro λ . O y no corpo da segunda expressão é ligado ao segundo λ e o x é livre.

$$(\lambda x.x)(\lambda y.yx)$$

- Devemos tomar cuidado ao efetuarmos substituições para evitar misturar ocorrências livres e ligadas de um dado identificador. Na expressão abaixo, a função da esquerda contém um y ligado, enquanto o y da direita é livre.

$$(\lambda x. (\lambda y. xy))y$$

- Uma substituição incorreta misturaria os dois identificadores, dando um resultado errado:

$$\lambda y. yy$$

- Se, no entanto, renomearmos o y ligado por t obteremos o resultado correto:

$$(\lambda x. (\lambda t. xt))y = (\lambda t. yt)$$

- Uma linguagem de programação deve ser capaz de efetuar cálculos aritméticos.
- Números podem ser representados pela teoria dos sucessores de Peano.
- Assim o zero pode ser representado por

$$\lambda sz.z$$

- E sucessivamente, os primeiros números naturais podem ser definidos por:

$$1 \equiv \lambda sz.s(z)$$

$$2 \equiv \lambda sz.s(s(z))$$

$$3 \equiv \lambda sz.s(s(s(z)))$$

Função Sucessor (S)

A função sucessor pode ser definida por:

$$S \equiv \lambda wyx. y(wyx)$$

Função Sucessor (S): Exemplo

A função sucessor aplicada a zero dá:

$$S0 \equiv (\lambda wyx.y(\boxed{w}yx))(\lambda sz.z) =$$

$$\lambda yx.y(\boxed{(\lambda sz.z)}yx) =$$

$$\lambda yx.y(\boxed{(\lambda z.z)}x) =$$

$$\lambda yx.y(x) =$$

$$\lambda sz.s(z) \equiv 1$$

Agora que já sabemos como funciona o sucessor de um número natural em λ vamos calcular $S2$?

Função Sucessor (S)

A função sucessor aplicada a dois dá:

$$\begin{aligned} S2 &\equiv (\lambda w y x. y(\boxed{w} y x)) (\lambda s z. s(s(z))) = \\ &\lambda y x. y(\boxed{(\lambda s z. s(s(z)))} y x) = \\ &\lambda y x. y(\boxed{(\lambda z. y(y(z)))} x) = \\ &\lambda y x. y(y(y(x))) = \\ &\lambda s z. s(s(s(z))) \equiv 3 \end{aligned}$$

- A adição pode ser obtida do seguinte modo:

$$2S3 =$$

$$(\lambda sz.s(sz))S3 =$$

$$(\lambda z.S(Sz))3 =$$

$$SS3 = S4 = 5$$

- O verdadeiro pode ser definido como "escolhe primeiro" entre dois argumentos:

$$T \equiv \lambda xy.x$$

- O falso pode ser definido como "escolhe segundo":

$$F \equiv \lambda xy.y$$

- Podemos definir os operadores lógicos usando-os como representação de suas tabelas-verdade.
- A função *E* de dois argumentos pode ser definida como:

$$\wedge \equiv \lambda xy. xyF$$

- A função *OU* de dois argumentos pode ser definida como:

$$\vee \equiv \lambda xy. xTy$$

- A função *NOT* de um argumento pode ser definida como:

$$\neg \equiv \lambda x. xFT$$

- É muito conveniente que uma linguagem de programação tenha uma função que é verdadeira se um número é zero e falsa caso contrário.
- A função a seguir cumpre este papel:

$$Z \equiv \lambda x. xF \neg F$$

- A estratégia para encontrar o predecessor de um número será criar um par $(n, n - 1)$ e escolher o segundo elemento do par como resultado.
- Um par (a, b) pode ser representado em λ -cálculo como:

$$(\lambda z.zab)$$

- Para acessar o primeiro elemento do par basta aplicar esta função a T :

$$(\lambda z.zab)T = Tab = a$$

- Para acessar o segundo elemento do par basta aplicar esta função a F :

$$(\lambda z.zab)F = Fab = b$$

- A função a seguir gera a partir do par $(n, n - 1)$ o par $(n + 1, n)$:

$$\Phi \equiv (\lambda pz.z(S(pT))(pT))$$

- O predecessor de um número n é obtido pela por aplicar n vezes a função Φ ao par $(\lambda z.z00)$ e então selecionar o segundo membro do novo par:

$$P \equiv (\lambda n.(n\Phi(\lambda z.z00))F)$$

- Observe que o predecessor acima retorna zero como predecessor de zero.

- Usando a função predecessor podemos definir uma função que testa se um número x é maior ou igual a outro número y :

$$G \equiv (\lambda xy. Z(xPy))$$

- Para obtermos a igualdade observamos que se $x \geq y$ e se $y \geq x$, então $x = y$:

$$E \equiv (\lambda xy. \wedge (Z(xPy))(Z(yPx)))$$

- Funções recursivas podem ser definidas no λ -cálculo usando uma função que chama uma função y e então regenera a si mesma:

$$Y \equiv (\lambda y. (\lambda x. y(xx)) (\lambda x. y(xx)))$$

- A função Y aplicada a uma função R dá:

$$YR \equiv (\lambda x. R(xx))(\lambda x. R(xx))$$

$$\equiv R((\lambda x. R(xx))(\lambda x. R(xx)))$$

- Mas isto significa que

$$YR \equiv R(YR)$$

- Uma função que soma os n primeiros naturais de modo recursivo é dada por:

$$\sum_{i=0}^n i = n + \sum_{i=0}^{n-1} i$$

- Vamos usar a seguinte definição para R :

$$R \equiv (\lambda rn. Zn0(nS(r(Pn))))$$

Exemplo (cont.)

$$R \equiv (\lambda rn. Zn0(nS(r(Pn))))$$

- Vejamos quanto dá $YR3$:

$$YR3 \equiv R(YR)3 \equiv Z30(3S(YR(P3)))$$

- Como 3 não é igual a zero, a avaliação é equivalente a

$$3S(YR(P3))$$

- Ou seja, a soma de 0 a 3 é igual a 3 mais a soma dos números de 0 até o predecessor de 3 (que é 2).
- O resultado final será:

$$3S(2S(1S0)) \equiv 6$$